

FastAPI Endpoint 500 Error Resolution SOP

Objective

Restore API availability when a FastAPI endpoint returns HTTP 500 due to unhandled exceptions, invalid input handling, or backend failure.

Scope

Applicable to FastAPI-based services, backend APIs, API gateways, DevOps teams, QA teams, and platform teams supporting production, staging, or synthetic incident workflows.

Pre-requisites

- 1. Access to FastAPI application logs, deployment logs, and API gateway or ingress logs.
- 2. Access to monitoring dashboards such as Grafana, Prometheus, APM, or cloud monitoring.
- 3. Permission to view application configuration, environment variables, and deployment status.
- 4. API testing tool such as curl, Postman, Swagger UI, or automated smoke test scripts.
- 5. Escalation contact for backend, DevOps, database, and security teams.

Incident Metadata

Field	Value
Ticket ID	T003
Issue	FastAPI endpoint returning 500 error
Category	API
Service	FastAPI
Severity	High
Status	Resolved
Description	API fails when calling POST endpoint
Expected Resolution	Validate request schema and fix input validation logic

Related Log Evidence

Log ID	Timestamp	Level	Message	Error Code
L003	5/1/2026 10:01	ERROR	Unhandled exception in API	API_500
L024	5/1/2026 10:12	ERROR	Unhandled exception in API	API_500
L044	5/1/2026 10:22	ERROR	Unhandled exception in API	API_500

Procedure

- 1. Confirm the endpoint, HTTP method, request payload, timestamp, and affected users.
- 2. Review application logs for stack trace, exception type, and correlation ID.
- 3. Validate the request body against the Pydantic input schema and required headers.
- 4. Check dependent services such as database, cache, message broker, or downstream API.
- 5. Reproduce the failure in a lower environment or with a safe test payload.
- 6. Apply schema validation, exception handling, or dependency fix as required.
- 7. Run smoke tests for the impacted endpoint and adjacent endpoints.
- 8. Monitor HTTP 5xx rate and update the ticket with root cause and resolution evidence.

Common Commands

```
curl -i -X GET http://<host>/health
curl -i -X POST http://<host>/<endpoint> -H "Content-Type: application/json" -d @payload.json
kubectl get pods -n <namespace> | grep fastapi
kubectl logs <fastapi-pod> -n <namespace> --tail=100
kubectl describe pod <fastapi-pod> -n <namespace>
```

Troubleshooting Matrix

Issue	Possible Cause	Resolution
HTTP 500 returned	Unhandled exception	Review stack trace and fix code path
POST endpoint failing	Invalid request schema	Validate Pydantic model and payload
Intermittent 500	Downstream dependency issue	Check database or external API health
No stack trace visible	Logging gap	Enable structured exception logging

Best Practices

1. Use global exception handlers for controlled error responses.
2. Add request validation tests for critical POST endpoints.
3. Log correlation IDs without exposing sensitive data.
4. Track HTTP 5xx rate by endpoint and release version.

References

FastAPI Docs | Internal Logs CSV | Tickets CSV | Monitoring Dashboards | API Gateway Logs | Backend Runbook